

System Requirements Specification

1. Introduction

1.1 Purpose

Repo Security Auditor is a tool that analyzes GitHub repositories for common security issues and presents the results in a clear security report.

The system focuses on helping developers identify potential vulnerabilities, exposed secrets, vulnerable dependencies, and insecure configurations before deployment.

1.2 Objective

The main objective is to provide a simple workflow:

GitHub Repository → Security Scan → Findings → Security Report

The MVP will focus on a limited number of security checks rather than attempting to detect every possible vulnerability.

2. System Scope

2.1 In Scope

The MVP will support:

- User authentication
- GitHub repository submission
- Repository validation
- Security scanning
- Source-code security checks
- Secret detection
- Dependency vulnerability checks
- Basic configuration checks
- Finding severity and confidence
- Security reports
- Scan history

2.2 Out of Scope

The MVP will not include:

- Automated exploitation
- Penetration testing

- Runtime monitoring
- Network vulnerability scanning
- Automatic code fixes
- Enterprise security management

3. Users

Developer

Submits repositories, runs scans, and reviews security findings.

Security Analyst

Reviews findings and helps validate potential security issues.

Administrator

Manages users and basic system configuration.

4. Functional Requirements

FR-01: Authentication

Users must be able to authenticate before accessing protected system features.

FR-02: Repository Submission

Authenticated users must be able to submit a GitHub repository for scanning.

FR-03: Repository Validation

The system must verify that the submitted repository exists and can be accessed before starting a scan.

FR-04: Security Scan

The system must analyze the repository using the security checks supported by the MVP.

These checks should include:

- Selected source-code vulnerabilities
- Exposed secrets
- Vulnerable dependencies
- Selected insecure configurations

FR-05: Scan Status

The system must show the status of a scan, including whether it is queued, running, completed, or failed.

FR-06: Findings

The system must generate findings for detected security issues.

Each finding should contain, where available:

- Title
- Severity
- Confidence
- Description
- File and line
- Evidence
- Recommended action

FR-07: Finding Review

Users must be able to view and review findings from a completed scan.

FR-08: Security Report

The system must generate a report summarizing the scan results and detected issues.

FR-09: Scan History

Users must be able to view previous scans for repositories they are authorized to access.

FR-10: Access Control

Users must only be able to access repositories, scans, findings, and reports they are authorized to view.

5. Non-Functional Requirements

NFR-01: Security

Repository contents must be treated as untrusted input and analyzed in an isolated environment.

NFR-02: Performance

Long-running scans must run separately from normal user requests so that the application remains responsive.

NFR-03: Reliability

A failure in one security check should not unnecessarily discard results from other completed checks.

NFR-04: Resource Limits

The system must limit resources used by individual scans, including execution time and storage.

NFR-05: Extensibility

The scanning system should allow additional security checks to be added without redesigning the entire application.

NFR-06: Usability

Findings should clearly explain what was detected, where it was detected, and why it matters.

6. Data Requirements

The system will maintain information for:

- User
- ID
- Role
- Authentication information
- Repository
- ID
- URL
- Owner
- Scan
- ID
- Repository
- Status
- Date/time
- Finding
- ID
- Scan
- Category
- Severity
- Confidence
- Location

- Evidence
- Recommendation
- Report
- Scan
- Summary
- Findings

7. Constraints and Assumptions

- The MVP will focus on GitHub repositories.
- Only selected programming languages and vulnerability types will be supported.
- Dependency analysis depends on an available vulnerability database.
- Users are responsible for having permission to scan submitted repositories.
- Automated scanning may produce false positives and does not guarantee that a repository is completely secure.
- The MVP focuses on static analysis rather than active exploitation.

8. MVP Acceptance Criteria

The MVP is complete when a user can:

- Authenticate.
- Submit a GitHub repository.
- Start a security scan.
- View scan progress and status.
- Detect the security issues supported by the MVP.
- View findings with relevant evidence and severity.
- View a security report.
- Review previous scans.
- Access only scans and reports they are authorized to view.